

PC/GEOS 1.x SDK 0.1

Experimental Linux and Windows development kit

This manual accompanies PC/GEOS 1.x SDK 0.1. It documents the supported build workflow, native UI reconstruction, target compatibility, public C API, host setup, file/dialog techniques, geode structure, reverse-engineering lessons, porting guidance, and troubleshooting.

Status: experimental. The SDK is validated structurally against a known GeoWorks Pro 1.2 target. Generated applications still require runtime testing in a recoverable Ensemble installation.

Getting Started

1. Install the host toolchain

Linux

Install Python 3.10+ plus either:

- GCC with `-m16`, GNU `ld`, `nm`, and `objcopy`, or
- LLVM/Clang, LLD, `llvm-nm`, and `llvm-objcopy`.

Run:

```
./bin/pcgeos1 doctor
```

Windows

The native Windows path uses LLVM because MinGW binutils do not reliably provide the ELF link/extract workflow used by this SDK.

Install:

- Python 3.10 or newer
- LLVM for Windows, including `clang.exe`, `ld.lld.exe`, `llvm-nm.exe`, and `llvm-objcopy.exe`

Ensure both Python and LLVM are on PATH, then run:

```
.\bin\pcgeos1.ps1 doctor
```

WSL2 is also supported as a practical alternative: use the Linux instructions inside WSL.

2. Supply a compatible target archive

SDK 0.1 does not redistribute GeoWorks binaries. Point builds at your own GWP2.zip containing the exact target applications and libraries.

Verify it:

```
./bin/pcgeos1 scan-target --gwp /path/to/GWP2.zip --output target.json
```

The basic UI builder validates four template files by SHA-256. See TARGET_COMPATIBILITY.md.

3. Create a project

```
./bin/pcgeos1 new hello2 --name "Hello Two" --permanent-name hello2 --token HL02  
cd hello2
```

The generated project contains:

```
project.json
src/app.c
src/help.txt
```

4. Build

Linux auto-detection:

```
/path/to/pcgeos1-sdk-0.1/bin/pcgeos1 build project.json --gwp /path/to/GWP2.zip
```

Windows LLVM:

```
C:\path\pcgeos1-sdk-0.1\bin\pcgeos1.ps1 build project.json --gwp C:\path\GWP2.zip --toolcha
```

The output `.GEO` appears in the project directory. Detailed intermediate files and `build-report.json` appear under `build/`.

5. Inspect the result

```
./bin/pcgeos1 inspect hello2/HELL02.GEO
```

Confirm:

- file format 1
- kernel protocol 622.3
- ui protocol 693.0
- expected permanent name and application token

6. Test in GEOS

Use a disposable or snapshotted installation.

1. Exit GEOS.
2. Copy the generated `.GEO` into a test `WORLD` directory.
3. Restart GEOS.
4. Launch it from GeoManager.
5. Test mouse, keyboard, redraw, title-bar Help, application close, and repeated launch.
6. If anything fails, record the exact error and restore the snapshot.

The SDK's host tests cannot certify loader/UI behavior in a real Ensemble session.

PC/GEOS 1.x SDK 0.1 Manual

Purpose

SDK 0.1 makes it possible to create experimental native PC/GEOS 1.x applications from modern Linux and Windows development machines without requiring the original historical compiler suite. It does this by combining modern free-standing x86 compilation with exact native object templates recovered from a known target installation.

This is deliberately a small, auditable SDK rather than a claim to recreate the complete GeoWorks development environment.

Architecture

The build has five stages:

1. **Target verification** - hashes and parses native applications from the user-supplied archive.
2. **C compilation** - compiles application code in a freestanding 16-bit execution mode for a 386+ CPU.
3. **Native ABI bridge** - handwritten assembly enters/exits GEOS correctly and periodically restores a scheduler-safe native context.
4. **UI reconstruction** - builds local-memory object resources from exact 1.x templates.
5. **Geode packaging** - writes resources, imports, protocols, and relocation records in GEOS 1.x format.

Why a target archive is required

The original generic UI object compiler and matching 1.x headers are not part of this SDK. Class instance layouts and private hints are therefore taken from exact native applications. SDK 0.1 uses:

- SPINTEXT.GEO for the process/application/primary/view skeleton
- VIEWER.GEO for title-bar Help and scrollable Help
- NOTEPAD.GEO and TERM.GEO in the advanced MiniCalc reference

This makes the basic shell look and behave like a native application, but it is version-specific.

Modern compiler model

-m16 does not turn GCC or Clang into an 8086 small-model compiler. The compiler still assumes 32-bit C arithmetic/register conventions, while the assembler emits instructions for a 16-bit code segment. Therefore:

- SDK-generated C applications require a 386 or newer CPU.
- C pointers are not general GEOS far pointers.

- GEOS services should be accessed through assembly adapters rather than called directly from C.
- Application state lives in a private appended resource with a private stack.

Scheduler bridge

Instrumented C calls `native_poll` at basic blocks and after at most eight straight-line instructions. `native_poll` saves full 32-bit state, restores the real GEOS stack, enables interrupts briefly, then returns to the private application stack.

This was developed to avoid assuming that a 1.x scheduler preserves upper halves of 386 registers used by a modern compiler.

Project file

`project.json` fields:

- `long_name`: user-visible application name, up to the geode long-name field limit.
- `permanent_name`: 1-8 ASCII characters; permanent geode identity.
- `token`: exactly four ASCII characters.
- `output`: output filename, normally uppercase `.GEO`.
- `release`: four 16-bit release numbers.
- `protocol`: two 16-bit application protocol numbers.
- `viewport`: `[width,height]` for the basic GenView shell.
- `sources`: C source files relative to the project.
- `help`: ASCII help text file.
- `native_stack_bytes`: native process stack requested in the geode header.
- `cflags`: optional additional compiler flags.
- `allow_unknown_target`: defaults false. Do not enable casually.

Application callback API

The basic template implements four callbacks:

```
void app_init(void);
void app_draw(void);
void app_key(void);
void app_click(void);
```

`app_draw` emits a list of simple draw commands. Pointer and keyboard callbacks inspect globals declared in `pcgeos1.h`.

Native drawing subset

SDK 0.1 exposes rectangles and text:

```
pcg_begin_frame();
pcg_rect(x1,y1,x2,y2,color);
pcg_text(x,y,color,"Hello");
```

The assembly renderer maps these to recovered kernel graphics entry points. This is intentionally a small surface that can be audited against the known target.

Redraws

Call `pcg_invalidate()` after state changes. When a pointer or handled keyboard callback returns, the native adapter redraws the view if requested.

Keyboard input

The basic `GenView` has the native 1.x send-control-characters bit enabled. `pcg_key_char` and `pcg_key_flags` receive the native message fields. Set `pcg_key_handled=1` if the application consumes the key; otherwise the message is forwarded to the superclass.

Pointer input

`pcg_pointer_x`, `pcg_pointer_y`, and `pcg_pointer_info` receive the selector message values. The basic SDK does not yet normalize button/modifier bits into a higher-level portable event structure.

Help

The project shell uses:

- the Viewer's native title-bar Help trigger and hint
- the Viewer's scrollable `GenTextDisplay` help dialog
- native Page Up, Page Down, and Close controls

The help text is embedded in the UI resource at build time.

File I/O and complex dialogs

The basic runtime does not expose file I/O yet. The advanced `MiniCalc` reference contains a tested design for:

- native `FileOpen`, `FileCreate`, `FileRead`, `FileWrite`, `FileClose`, rename and delete wrappers
- directory push/pop behavior
- `GenFileSelector`-based native Open and Save As dialogs
- filename fields and reply bars
- transactional save/rollback logic

See `FILE_IO_AND_DIALOGS.md` and `examples/minicalc/`.

Relocations

Assembly labels beginning `krel_` identify kernel far-call locations. Labels beginning `rrel_` identify resource references. The packer converts these symbol locations into GEOS relocation records.

The generic runtime currently uses only recovered kernel ordinals required for object dispatch, graphics, and the application superclass path.

UI development strategy

There are three safe levels:

1. **Basic shell** - supported by SDK 0.1; use the generated native view and Help.
2. **Template composition** - copy exact controls from known target applications and modify generic tree links, monikers, actions, hints, dimensions and text.
3. **New class layouts** - not supported. Do not invent private instance structures without evidence.

Debugging

Host-side debugging consists of:

- `pcgeos1 inspect` for geode headers/imports/resources
- `scan-target` for target inventory
- `build-report.json` for symbols, hashes and instrumentation
- `app.map` and `symbols.txt` for code offsets
- disassembly of `text.bin` with `objdump` or LLVM tools

Runtime debugging requires a DOS/GEOS test environment. Record loader messages, exact screen state, and whether failure occurs before or after the primary window appears.

ABI stability policy for SDK 0.1

There is no stable binary ABI across SDK versions yet. Source compatibility is the goal, but the runtime, project schema, object templates and C API may change in 0.x releases. Pin SDK 0.1 for reproducible work.

Build System Internals

pcgeos1 build

The generic builder is implemented by `pcgeos1/buildsys.py`.

Inputs

- `project.json`
- project C sources
- SDK runtime sources
- user-supplied target archive

Target stage

`pcgeos1/target.py` opens the archive without extracting arbitrary paths and loads exactly identified members. Known template hashes are checked by default.

Compilation stage

Each C source is compiled to assembly rather than directly to an object. This deliberate intermediate step lets the SDK instrument control flow with `native_poll` calls.

The transformed assembly is then assembled into 32-bit ELF relocatable objects. The handwritten native adapter is assembled separately.

Link stage

`runtime/native.ld` creates an ELF intermediate with:

- `.text` beginning at offset 0;
- `.data` beginning at private-engine offset 0x100;
- `.bss` following `.data` but not stored in the ELF binary extraction;
- `__engine_used` marking the end of initialized plus uninitialized engine state.

The linker asserts that code and engine state leave suitable 64 KiB headroom.

Extraction stage

The builder extracts raw `.text` and `.data`. It allocates a 65,520-byte private engine resource, writes a small SDK tag at the start, and places initialized C data at offset 0x100. BSS starts zero because the resource is created zero-filled.

Symbol stage

`nm/llvm-nm` output is parsed to find:

- process entry points;
- `__engine_used`;
- `krel_*` kernel relocation sites;
- `rrel_*` resource relocation sites.

Packaging stage

`pcgeos1/packer.py` starts from the known 1.x skeleton. It preserves the native import table and unchanged resources, replaces `dgroup/code/UI` resources, appends the engine resource, recalculates file positions, and writes relocation records.

The finished file is reparsed immediately and checked for:

- GEOS 1.x format;
- unchanged import table;
- unchanged kernel protocol;
- valid resource/relocation bounds.

Build evidence

The builder writes:

```
build/build-report.json
build/app.map
build/symbols.txt
build/*.s
build/*.o
build/app.elf
build/text.bin
build/data.bin
```

Keep these files when reporting a runtime failure.

SDK Command-Line Reference

pcgeos1 doctor

Reports Python/platform information and available GNU/LLVM toolchains.

pcgeos1 new DIRECTORY [options]

Creates a new basic native-view project.

Options:

```
--name "Long Name"  
--permanent-name name8  
--token ABC1
```

The permanent name is truncated to eight characters; choose a unique value yourself. The token must represent four characters.

pcgeos1 build [project.json] --gwp ARCHIVE

Builds the project.

Options:

```
--toolchain auto|gnu|llvm  
--build-dir PATH
```

auto prefers GNU on non-Windows hosts when available, otherwise LLVM.

pcgeos1 inspect FILE.GEO

Prints a compact JSON summary including format, names, release/protocol, imports, resource count, export count and relocation count.

pcgeos1 scan-target --gwp ARCHIVE [--output target.json]

Parses every recognized .GEO in the archive and reports protocols/imports/resources. This is useful when developing a new target profile.

Python modules

pcgeos1.geode

Bounds-checked GEOS parser plus resource/relocation access helpers.

pcgeos1.target

Known target hashes, safe archive-member lookup and target inventory.

pcgeos1.ui

LMEM and basic native UI composition helpers.

pcgeos1.toolchain

GNU/LLVM toolchain abstraction.

pcgeos1.instrument

Compiler-generated assembly instrumentation for scheduler-safe polling.

pcgeos1.packer

GEOS 1.x resource/header/relocation packer based on a known native skeleton.

pcgeos1.buildsys

Generic project build orchestration.

Experimental C API

Header: `include/pcgeos1.h`

The API is intentionally small in SDK 0.1.

Types

`pcg_u8`, `pcg_u16`, `pcg_u32`, `pcg_i16` are fixed-width-style aliases for the compiler model used by the SDK.

Do not pass C pointers directly to arbitrary GEOS APIs. Native services use segmented addresses and calling conventions that must be bridged explicitly.

Application callbacks

```
void app_init(void);  
void app_draw(void);  
void app_key(void);  
void app_click(void);
```

app_init

Called once after the 386 CPU probe succeeds during application open.

app_draw

Called before the native view is redrawn. Emit drawing commands using the functions below.

app_key

Called for native keyboard-character messages delivered to the view. Inputs:

```
pcg_key_char  
pcg_key_flags
```

Set:

```
pcg_key_handled = 1;
```

to consume the message.

app_click

Called for the view selection message. Inputs:

```
pcg_pointer_x  
pcg_pointer_y  
pcg_pointer_info
```

Drawing

```
void pcg_begin_frame(void);  
void pcg_rect(int x1,int y1,int x2,int y2,pcg_u16 color);  
void pcg_text(int x,int y,pcg_u16 color,const char *text);  
void pcg_invalidate(void);
```

`app_draw()` is automatically preceded by `pcg_begin_frame()` by the SDK dispatcher.

`pcg_invalidate()` requests a redraw after the current input callback returns to the native GEOS context.

Colors

The current renderer exposes a small set of indexed-color constants. These are target-era GEOS color values, not RGB.

Limits

- 96 drawing commands per frame.
- Text commands store at most 63 characters plus terminator.
- The basic renderer supports only filled rectangles and text.
- No GState or graphics handle is exposed to C in SDK 0.1.

Native UI Reconstruction

PC/GEOS generic UI objects contain class-specific binary instance data. SDK 0.1 does not invent those layouts. Instead it extracts complete object chunks from native target applications and changes only fields that have been verified through comparison and runtime-oriented work.

Basic shell

The supported shell contains:

- application object
- primary window
- GenView
- native title-bar Help trigger
- native scrollable Help dialog

The view size is configured by `project.json`.

Local-memory resources

`pcgeos1/ui.py` constructs a local-memory resource with:

- LMEM header
- handle table
- object/data chunks
- object flags
- 4-byte chunk alignment
- 16-byte resource padding

The code updates generic links for parent, next sibling, first child, moniker and hints while retaining unknown class-specific bytes from the native template.

Advanced controls

The MiniCalc reference demonstrates recovered native templates for:

- menus
- GenTriggers
- GenText input fields
- read-only GenText displays
- GenFileSelector
- modal dialogs
- reply bars and default/cancel buttons
- title-bar Help

Developers can adapt `examples/minicalc/source-package/TOOLS/ui_builder.py`, but this is an advanced, target-specific path. Create a new target profile rather than assuming offsets are universal.

Rule of thumb

If you cannot explain which native template supplied an object's class-specific bytes, do not ship an invented binary instance.

File I/O and Native File Dialogs

The basic SDK 0.1 runtime intentionally omits file services from the public C API. The advanced MiniCalc example contains the recovered design used successfully in its modeled instruction tests.

Native file services used by the advanced example

The adapter wraps target kernel entry points for:

- open
- create
- read
- write
- close
- rename
- delete
- push directory
- pop directory

All services execute on the original native GEOS stack with interrupts enabled. The private C register state is saved before crossing into GEOS.

Why that matters

Calling kernel file services while leaving GEOS on the private GCC stack would expose scheduler/file code to a stack layout that the operating system did not create. The advanced adapter therefore restores the real session stack first.

Native Open and Save As

MiniCalc reconstructs the Viewer's native GenFileSelector dialogs. The selector owns directory and drive browsing. Selection notifications populate application fields; actual save occurs only from the explicit Save trigger.

A useful safety pattern is to keep three concepts separate:

1. directory currently being browsed;
2. currently opened document path;
3. user-entered destination filename.

Cancelling a dialog must not silently change the document's save directory.

Transactional saving

The advanced example writes temporary master/view files, closes and validates them, renames previous versions to backups, installs new versions, and attempts rollback after failure.

This pattern is strongly recommended for old systems with limited filesystem guarantees.

Promoting file I/O into the public SDK

A future 0.2 should expose a small stable API only after live GEOS tests confirm the recovered ordinals and error mappings across the target profile.

GEOS Executable Format Notes

`pcgeos1/geode.py` is a bounds-checked reader for the GEOS 1.x and 2.x executable layouts encountered during this work. It is not a loader.

GEOS 1.x

The known 1.x geodes use signature bytes:

`C7 45 CF 53`

The executable header starts at byte 200. Resource file positions are absolute.

GEOS 2.x

The observed 2.x files use:

`C7 45 C1 53`

The executable header starts at byte 256, and stored resource positions require the 256-byte header origin adjustment.

Tables

The parser reads:

- imported libraries
- exported entry table
- resource size table
- resource file-position table
- relocation-size table
- resource flags
- relocation records

It verifies duplicate count fields, resource bounds, overlap, import indices and relocation target bounds.

Relocation records

Each observed relocation record is four bytes:

`info, extra, target_offset`

The upper nibble of `info` identifies the source class; the lower nibble identifies the relocation type. SDK 0.1 emits kernel far-call and resource relocations needed by its runtime.

Packer strategy

The SDK starts from a known native 1.x application skeleton, preserves its import table and compatible resources, replaces dgroup/code/UI resources, appends a private engine resource, and rewrites the resource descriptor table and payload positions.

This is safer than manufacturing every undocumented header field from zero in the first SDK release.

Target Compatibility

SDK 0.1 is validated against one GeoWorks Pro 1.2 target set.

Required native interface

- GEOS 1.x executable format
- kernel protocol: 622.3
- UI library protocol: 693.0
- basic application skeleton: SPINTEXT.GEO

Exact template hashes

File	SHA-256
SPINTEXT.GEO	848f5afb43a2184b01d2a4ad3231957b6ab9936006b23b4b2a260d122a
NOTEPAD.GEO	edc48848ff4b9d2880e715bbb29532f14b3b54293c45159c467e33e2a1
VIEWER.GEO	3d433a39c455a9cea242763dad0e3f681b4196fb7b1a75a08ca47e1aa8
TERM.GEO	2d66ac2ff9fd24c4591303b2cba7cec6f40f0d52b4f3d75d9e0cdaa5f

The generic basic-view build currently requires all four hashes even though it directly consumes only SPINTEXT and VIEWER. This is intentional: the target set is treated as a coherent known distribution.

Unknown targets

`allow_unknown_target` exists for research, not routine application development. Disabling hash checks does not make private object layouts compatible.

To add a target properly:

1. scan every geode and record kernel/UI protocols;
2. identify native applications that contain the required object types;
3. recover exact object chunks and hints;
4. verify superclass/message behavior;
5. add target-specific hashes and tests;
6. keep target profiles separate if layouts differ.

Linux Host Setup

GNU path

Required commands:

```
gcc
ld
nm
objcopy
python3
```

The GCC build uses `-m16`, assembles the generated source as 32-bit ELF objects, links with `ld -m elf_i386`, then extracts raw `.text` and `.data` sections.

Check:

```
./bin/pcgeos1 doctor
```

Build with an explicit GNU toolchain:

```
./bin/pcgeos1 build project.json --gwp /path/GWP2.zip --toolchain gnu
```

LLVM path

LLVM is also supported on Linux and is useful for matching the Windows build path:

```
clang
ld.lld
llvm-nm (GNU nm is accepted on Linux as a fallback)
llvm-objcopy
```

```
./bin/pcgeos1 build project.json --gwp /path/GWP2.zip --toolchain llvm
```

GNU and LLVM outputs are not expected to be byte-identical because code generation differs. Their geode interfaces and resource invariants should match.

Windows Host Setup

Native Windows build with LLVM

Install Python 3.10+ and a current LLVM distribution containing:

```
clang.exe  
ld.lld.exe  
llvm-nm.exe  
llvm-objcopy.exe
```

Add both Python and LLVM `bin` directories to `PATH`.

From PowerShell:

```
.\bin\pcgeos1.ps1 doctor  
.\bin\pcgeos1.ps1 build project.json --gwp C:\GEOSDEV\GWP2.zip --toolchain llvm
```

The SDK tells Clang to target `i386-unknown-none-elf`, so it produces ELF objects rather than Windows PE/COFF application objects. LLD links those objects as `elf_i386`; the SDK then extracts the raw code/data and creates the GEOS file itself.

Why not ordinary MinGW linking?

The SDK requires an ELF intermediate and explicit raw-section extraction. A normal MinGW link produces Windows-format output and does not match this pipeline. LLVM provides a consistent cross-platform path.

WSL2 alternative

If native LLVM setup is inconvenient, install a Linux distribution under WSL2 and use the Linux GNU instructions. The resulting `.GEO` is independent of the host OS.

Path notes

- Quote paths containing spaces.
- Keep project sources and the SDK on a normal local filesystem where possible.
- `project.json` uses forward or backward slash paths accepted by Python.

Validation status in SDK 0.1

The LLVM build path was exercised end-to-end with Clang/LLD/LLVM objcopy producing the same GEOS structure as the GNU build, but that validation host was Linux. The PowerShell launcher and Windows executable-name detection are included for native Windows use; native Windows execution of the

build driver should still be treated as release-candidate support until a Windows machine reproduces the Hello self-test.

Porting Existing Code to SDK 0.1

Good candidates

Applications with:

- self-contained calculations/data processing;
- simple native view drawing;
- keyboard and pointer interaction;
- modest memory requirements;
- 386+ target hardware.

Harder candidates

Applications heavily dependent on undocumented class instance data, multiple imported libraries, object inheritance, dynamic object creation, text-engine internals, or later GEOS APIs.

Recommended migration

1. Isolate platform-independent logic in ordinary C.
2. Replace libc dependencies with bounded local helpers or SDK functions.
3. Use `app_init`, `app_draw`, `app_key`, and `app_click` first.
4. Keep state in the private engine resource.
5. Add native services one at a time through assembly wrappers.
6. Recover UI controls from exact target applications before introducing them.
7. Add host tests for each state transition and file format.
8. Boot-test each milestone in a recoverable GEOS environment.

Memory model cautions

Avoid assuming that arbitrary GEOS segments can be represented by C pointers. Use explicit copied buffers or native adapters. Keep individual geode resources under 64 KiB.

Adding Another PC/GEOS 1.x Target Profile

SDK 0.1 intentionally locks native object templates to a known GeoWorks Pro 1.2 build. Supporting another Ensemble/GeoWorks 1.x release should be done as a new profile, not by weakening validation.

Procedure

1. Run `pcgeos1 scan-target` and save the full inventory.
2. Identify the kernel protocol and generic UI library protocol.
3. Locate a small native application with a primary window and GenView.
4. Locate native examples of every additional control you need: menu, trigger, text field, file selector, interaction, Help, and so on.
5. Hash those donor binaries and preserve an unmodified archive.
6. Compare resource layouts and object chunks against the existing profile.
7. Disassemble message handlers to confirm message numbers and calling conventions where they cross into custom code.
8. Add profile-specific hashes and chunk metadata instead of changing the existing profile in place.
9. Build a minimal Hello application and validate structure.
10. Boot-test open, redraw, keyboard, pointer, Help, close, repeated launch, and low-memory behavior.

Evidence expected for a contributed profile

- target release identity and archive hash;
- kernel/UI protocols;
- hashes of every donor binary;
- object chunk provenance;
- build self-test output;
- real GEOS runtime test notes;
- any differences from the 622.3 / UI 693.0 profile.

Reverse Engineering Notes

The SDK was derived from several concrete PC/GEOS 1.x/2.x investigations.

Findings that shaped the SDK

- 1.x and 2.x geode containers are structurally different; changing a signature or protocol field is not a valid backport.
- Specific UI and generic UI protocols are meaningful ABI boundaries.
- Imported function ordinals must be validated against the exact target export table; an in-range number still does not prove semantic compatibility.
- Native UI object instance layouts are safer to recover from matching target applications than to guess from later source revisions.
- Native Help in the target can be built from the Viewer's title-bar trigger/hint plus its scrollable help dialog; it is not necessary to imitate Help in application drawing.
- Native GenFileSelector controls provide correct drive/directory behavior and should be preferred to custom browser widgets.
- Modern `-m16` compilation is practical for 386+ systems but needs a carefully isolated native ABI bridge.
- File services should execute on the original GEOS stack, not a compiler-private stack.
- Reproducible builds, input hashes and relocation audits are essential when the historical SDK is not used.

Later public source trees

Public PC/GEOS source code is valuable for understanding structures and semantics, but it may represent later releases. SDK 0.1 therefore treats the exact target binaries as authoritative for private UI layouts and recovered message/interface details.

Troubleshooting

No supported toolchain found

Run `pcgeos1 doctor`.

Linux: install GCC/binutils or LLVM.

Windows: install LLVM with Clang, LLD, `llvm-nm` and `llvm-objcopy`, or use WSL2.

Unsupported native ... template build

Your target archive does not match the SDK 0.1 profile. Do not simply disable the check for production. See `TARGET_COMPATIBILITY.md`.

Application does not appear in GeoManager

Check:

- `.GEO` is in the expected `WORLD` directory;
- only one copy with that identity is active;
- permanent name and token are unique;
- the target installation matches the expected kernel/UI protocols.

GEOS error during launch

Use `pcgeos1 inspect APP.GEO` and save `build/build-report.json`, `build/app.map`, and `build/symbols.txt`. Record the exact error before modifying the build.

Window appears but drawing is blank

Confirm `app_draw()` emits commands and that the view receives expose events. Compare with `examples/hello`.

Keyboard does not reach the application

The generic view must retain the `send-control-characters` flag. Do not replace the `GenView` template without checking that field.

Crashes only after some interaction

Suspect ABI crossings first. Do not call arbitrary kernel/UI ordinals directly from C. Keep native services behind an assembly adapter that restores the expected GEOS stack/context.

LLVM build works but differs from GCC

Different machine code is normal. Compare structural output with `pcgeos1 inspect`; do not require identical hashes across compilers.

Development Safety

This SDK creates low-level executable files for an old operating environment. Mistakes can hang or crash GEOS and can corrupt documents if file operations are wrong.

- Develop against copied installations or emulator snapshots.
- Never replace your only copy of a system library or UI geode.
- Keep generated applications in a separate test directory until they are stable.
- Make application file writes transactional where practical.
- Treat malformed or unknown geodes as untrusted input; use the bounds-checked parser.
- Do not disable target hash checks merely to make a build pass.
- Keep a DOS-level rollback path when testing native applications.